SCHOOL HOMEWORK - 2023

CLASS: INCIDENT RESPONSE AND DETECTION

SUBJECT: FILE EXTRACTION, FILE DECODING, DKIM, SILK, SURICATA

FRÉDÉRIC PERRON

Table of contents

Guideline

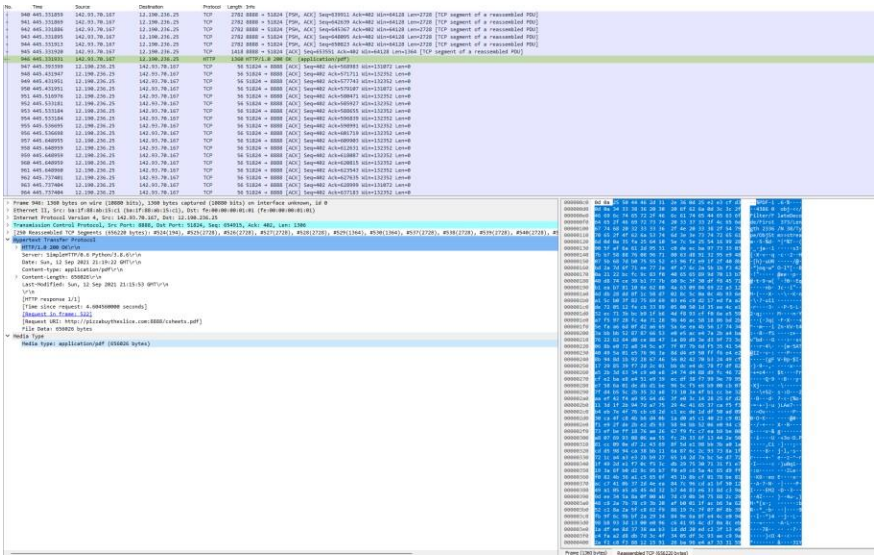
- All answers must be documented and supported by screenshots. Even if an answer is correct, if it is not documented, your grade will be **0 points**.
- You must demonstrate analysis and research in the answers you provide.
- Please submit your assignments in PDF or DOC format and present your answers under the questions below by copying them exactly, along with their question number.
- **Exercises 5 and 6 must include the username you created in Security Onion (not the student account).**

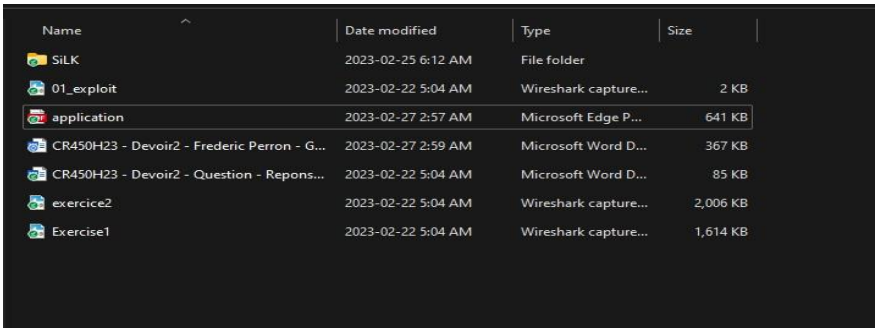
Exercise #1 : File extraction (10p)

In the file Exercise1.pcap you will find the same file .pdf transferred via HTML and via SMTP.

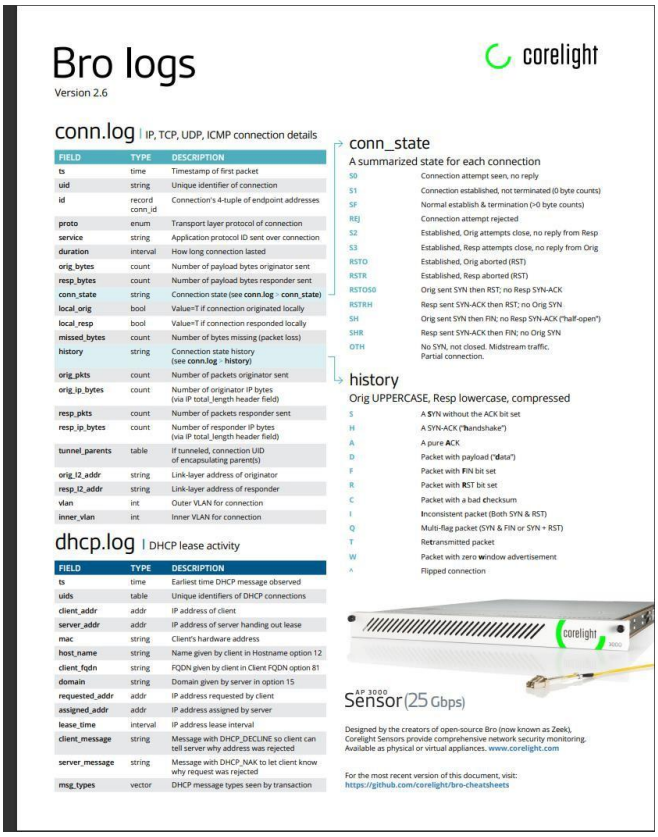
a) Extract the http file (using the stream) .(**2.5p**)

After identifying the HTTP packet containing application/pdf, I selected it and then clicked on "Export Specified Packets." I then saved it in my Assignment 2 folder under the name application.pdf. See images below.





Name	Date modified	Type	Size
SiLK	2023-02-25 6:12 AM	File folder	
01_exploit	2023-02-22 5:04 AM	Wireshark capture...	2 KB
application	2023-02-27 2:57 AM	Microsoft Edge P...	641 KB
CR450H23 - Devoir2 - Frederic Perron - G...	2023-02-27 2:59 AM	Microsoft Word D...	367 KB
CR450H23 - Devoir2 - Question - Repons...	2023-02-22 5:04 AM	Microsoft Word D...	85 KB
exercice2	2023-02-22 5:04 AM	Wireshark capture...	2,006 KB
Exercise1	2023-02-22 5:04 AM	Wireshark capture...	1,614 KB



Bro logs

Version 2.6

corelight

conn.log | IP, TCP, UDP, ICMP connection details

FIELD	TYPE	DESCRIPTION
ts	time	Timestamp of first packet
uid	string	Unique identifier of connection
id	record conn_id	Connection's 4-tuple of endpoint addresses
proto	enum	Transport layer protocol of connection
service	string	Application protocol ID sent over connection
duration	interval	How long connection lasted
orig_bytes	count	Number of payload bytes originator sent
resp_bytes	count	Number of payload bytes responder sent
conn_state	string	Connection state (see conn.log > conn_state)
local_orig	bool	Value=T if connection originated locally
local_resp	bool	Value=T if connection responded locally
missed_bytes	count	Number of bytes missing (packet loss)
history	string	Connection state history (see conn.log > history)
orig_pkts	count	Number of packets originator sent
orig_ip_bytes	count	Number of originator IP bytes (via IP total_length header field)
resp_pkts	count	Number of packets responder sent
resp_ip_bytes	count	Number of responder IP bytes (via IP total_length header field)
tunnel_parents	table	If tunneled, connection UID of encapsulating parent(s)
orig_l2_addr	string	Link-layer address of originator
resp_l2_addr	string	Link-layer address of responder
vlan	int	Outer VLAN for connection
inner_vlan	int	Inner VLAN for connection

dhcp.log | DHCP lease activity

FIELD	TYPE	DESCRIPTION
ts	time	Earliest time DHCP message observed
uids	table	Unique identifiers of DHCP connections
client_addr	addr	IP address of client
server_addr	addr	IP address of server handing out lease
mac	string	Client's hardware address
host_name	string	Name given by client in Hostname option 12
client_fqdn	string	FQDN given by client in Client FQDN option 81
domain	string	Domain given by server in option 15
requested_addr	addr	IP address requested by client
assigned_addr	addr	IP address assigned by server
lease_time	interval	IP address lease interval
client_message	string	Message with DHCP_DECLINE so client can tell server why address was rejected
server_message	string	Message with DHCP_NAK to let client know why request was rejected
msg_types	vector	DHCP message types seen by transaction

conn_state

A summarized state for each connection

S0	Connection attempt seen, no reply
S1	Connection established, not terminated (0 byte counts)
SF	Normal establish & termination (>0 byte counts)
REJ	Connection attempt rejected
S2	Established, Orig attempts close, no reply from Resp
S3	Established, Resp attempts close, no reply from Orig
RSTO	Established, Orig aborted (RST)
RSTR	Established, Resp aborted (RST)
RSTOS0	Orig sent SYN then RST; no Resp SYN-ACK
RSTRH	Resp sent SYN-ACK then RST; no Orig SYN
SH	Orig sent SYN then FIN; no Resp SYN-ACK ("half-open")
SHR	Resp sent SYN-ACK then FIN; no Orig SYN
OTH	No SYN, not closed. Midstream traffic. Partial connection.

history

Orig UPPERCASE, Resp lowercase, compressed

S	A SYN without the ACK bit set
H	A SYN-ACK ("handshake")
A	A pure ACK
D	Packet with payload ("data")
F	Packet with FIN bit set
R	Packet with RST bit set
C	Packet with a bad checksum
I	Inconsistent packet (Both SYN & RST)
Q	Multi-flag packet (SYN & FIN or SYN + RST)
T	Retransmitted packet
W	Packet with zero window advertisement
^	Flipped connection

AP 3000 Sensor (25 Gbps)

Designed by the creators of open-source Bro (now known as Zeek), Corelight Sensors provide comprehensive network security monitoring. Available as physical or virtual appliances. www.corelight.com

For the most recent version of this document, visit: https://github.com/corelight/bro-cheatsheets

Cover page of the PDF file extracted from http flux

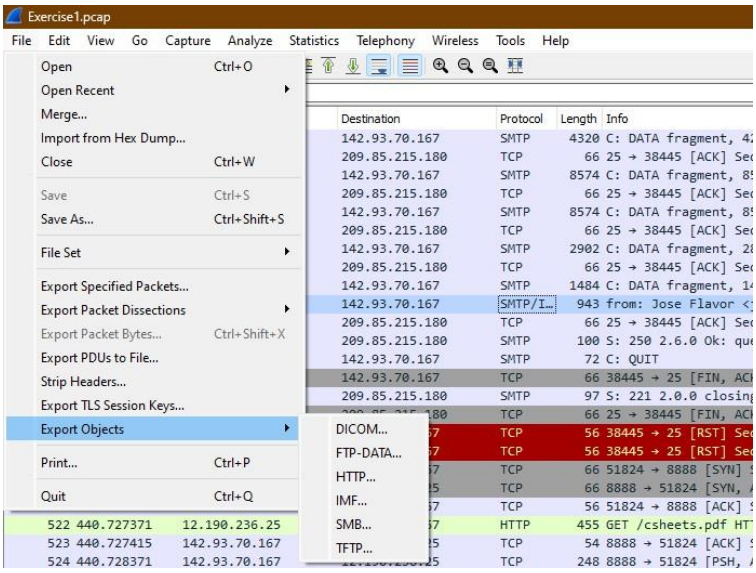
b) Extract the file attached to the SMTP traffic. (**5p**)

For this part of the exercise, I located the packet containing the SMTP/IMF (email) protocol. I noticed the application.pdf file and relevant information related to its extraction. I extracted the file by using Export Objects > IMF. After doing so, a new .eml file appeared in my Assignment 2 folder. After opening it, a window prompted me to open it with Mail, which I accepted. Once the email was opened, a PDF file was attached, which was the same one as in the previous exercise. *See the images below on the next page for the complete steps of exercise b).*

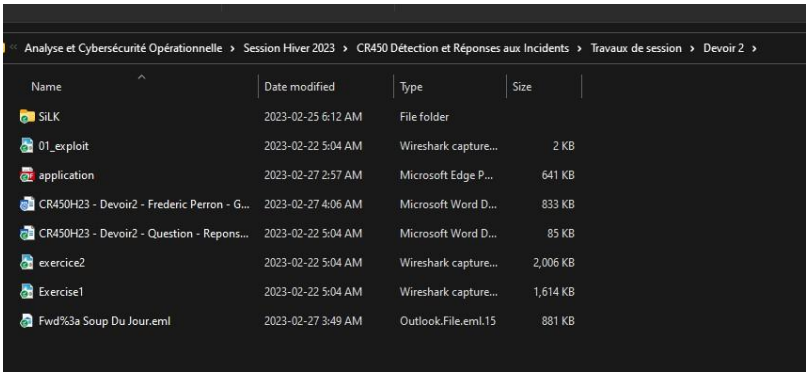
```
  509 0.861269    209.85.215.180   142.93.70.167    SMTP      1484 C: DATA fragment, 1418 bytes
  510 0.861331    209.85.215.180   142.93.70.167    SMTP/I…    943 from: Jose Flavor <joseflavor102030@gmail.com>, subj
  511 0.861333    142.93.70.167    209.85.215.180   TCP          66 25 → 38445 [ACK] Seq=278 Ack=901437 Win=1507200 Len=
```

```
3445 [ACK] Seq=278 Ack=899142 Win=1504384 Len=0 TSval=2122185812 TSecr=2749130397
A fragment, 1418 bytes
Jose Flavor <joseflavor102030@gmail.com>, subject: Fwd: Soup Du Jour,  (text/plain) (text/html) (application/pdf)
3445 [ACK] Seq=278 Ack=901437 Win=1507200 Len=0 TSval=2122185812 TSecr=2749130397
 2.6.0 Ok: queued as 68B5E2F4
```

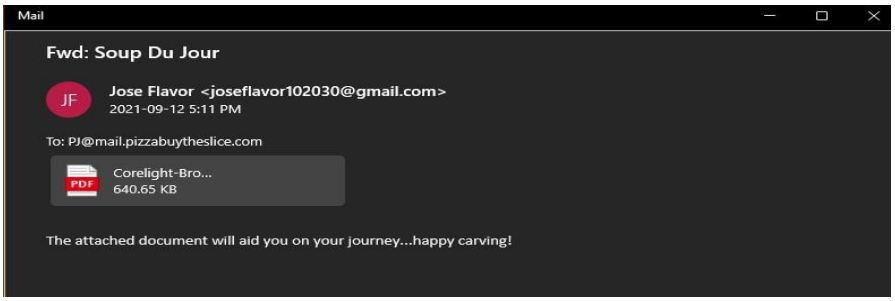
The packet SMTP/IMF in question



Extracted from IMF packet of Wireshark to my folder Devoir 2

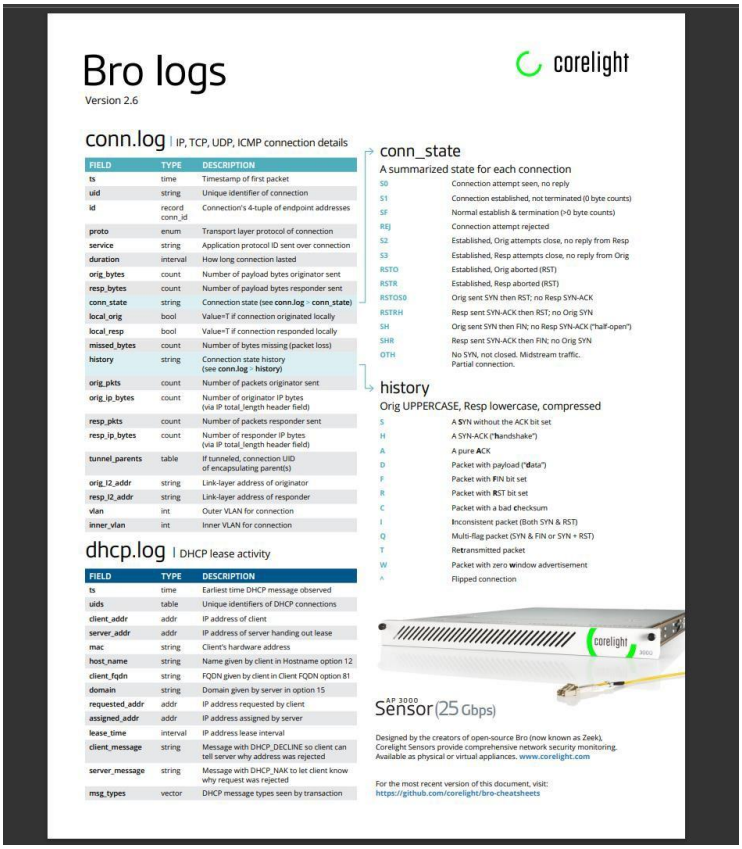


The file in question in my folder Devoir 2



The email containing the wanted PDF file

After opening the file, here is the page cover I got:



c) Compare the two files and state your conclusions. (**2.5p**)

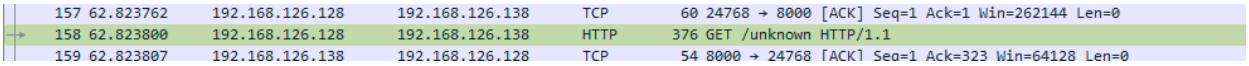
After comparing the two files, we can conclude that they are identical. The two files extracted from the two different packets (SMTP/IMF and HTTP) are the same "Bro logs" guide from Corelight.

Exercise #2 : File decoding (10p)

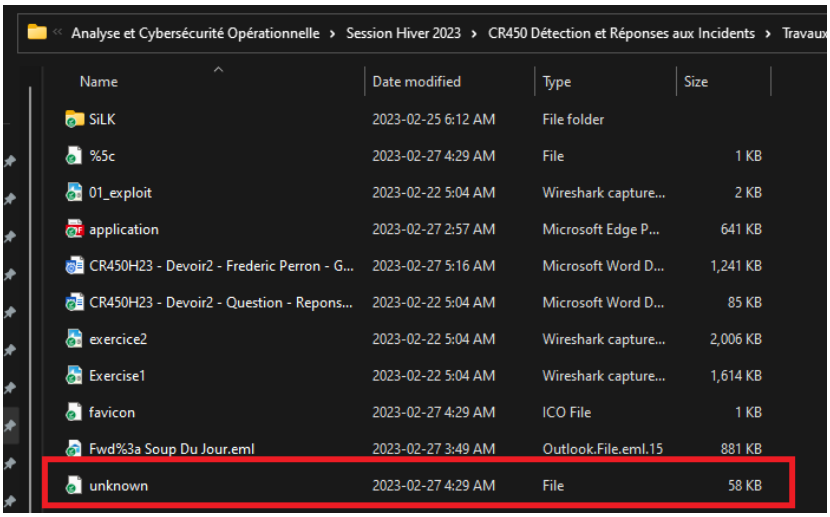
Analyze the file exercice2.pcap. The capture contains the transfer of several files. Proceed with extracting the file named **unknown** and try to convert it to its original format. Then answer the following question:

What type of file is it? Provide all the details you can find.

Hint: refer to the lab from Course 5



Packet with the file 'unknown' found

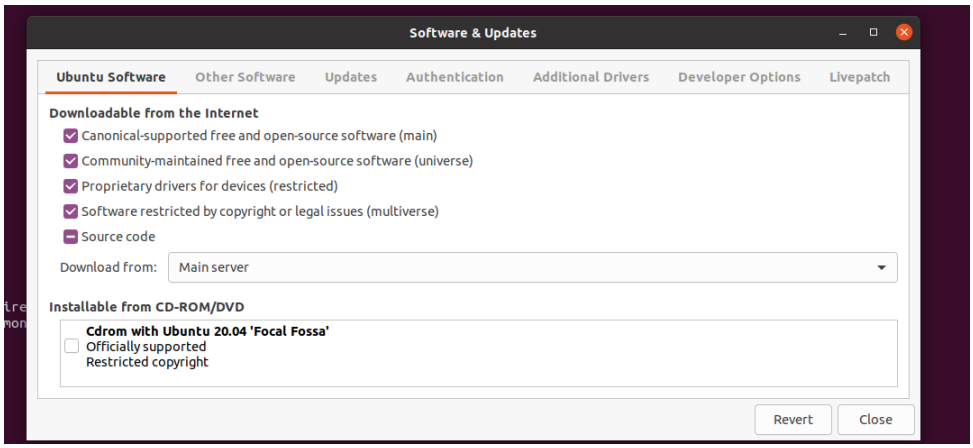


After exporting the file **unknown** into the same Assignment 2 folder, here is how I decoded it:

I first opened this file from my Security Onion VM rather than from my main/personal machine. I installed **bless**, and then in Software & Updates I changed the source code configuration to "Main Server" instead of the default "Server for Canada." Once that was done, I was able to install **yara** from the terminal. After installing yara, I ran **bless unknown** to open Bless along with the file to analyze.

To decode it, I executed **yara xorsigs.yar unknown -s** to obtain the signatures. I obtained **0x4e:$_13_d** as variables, in hexadecimal and decimal, etc. I then returned to Bless with the unknown file, selected everything with Ctrl+A, and entered the value **d** as hexadecimal using the Bitwise Operator.

I then obtained the same result, file type, and message as in the Lab from Course 5 ("MZ, this program cannot be run in DOS mode […]")



Change of source code from Server for Canada to Main Server

```
student@cr450:~/labs$ sudo apt install yara
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  docker-scan-plugin libfwupdplugin1 libxmlb1 ubuntu-advantage-desktop-daemon
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libyara3
The following NEW packages will be installed:
  libyara3 yara
0 upgraded, 2 newly installed, 0 to remove and 51 not upgraded.
Need to get 155 kB of archives.
After this operation, 486 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://ca.archive.ubuntu.com/ubuntu focal/universe amd64 libyara3 amd64 3.9.0-1 [135 kB]
Get:2 http://ca.archive.ubuntu.com/ubuntu focal/universe amd64 yara amd64 3.9.0-1 [20.4 kB]
Fetched 155 kB in 1s (111 kB/s)
Selecting previously unselected package libyara3:amd64.
(Reading database ... 186479 files and directories currently installed.)
Preparing to unpack .../libyara3_3.9.0-1_amd64.deb ...
Unpacking libyara3:amd64 (3.9.0-1) ...
Selecting previously unselected package yara.
Preparing to unpack .../yara_3.9.0-1_amd64.deb ...
Unpacking yara (3.9.0-1) ...
Setting up libyara3:amd64 (3.9.0-1) ...
Setting up yara (3.9.0-1) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9.9) ...
student@cr450:~/labs$ yara -v
3.9.0
```

downloading of YARA after changing the configuration of source code

```
student@cr450:~/labs$ sudo apt install bless --fix-missing
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  docker-scan-plugin libfwupdplugin1 libxmlb1 ubuntu-advantage-desktop-daemon
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libglade2-0 libglade2.0-cil libglib2.0-cil libgtk2.0-cil
Suggested packages:
  monodoc-gtk2.0-manual
The following NEW packages will be installed:
  bless libglade2-0 libglade2.0-cil libglib2.0-cil libgtk2.0-cil
0 upgraded, 5 newly installed, 0 to remove and 51 not upgraded.
Need to get 957 kB of archives.
After this operation, 4,217 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://ca.archive.ubuntu.com/ubuntu focal/universe amd64 libglade2-0 amd64 1:2.6.4-2ubuntu2 [44.3 kB]
Get:2 http://ca.archive.ubuntu.com/ubuntu focal/universe amd64 libglib2.0-cil amd64 2.12.40-3 [43.7 kB]
Get:3 http://ca.archive.ubuntu.com/ubuntu focal/universe amd64 libgtk2.0-cil amd64 2.12.40-3 [446 kB]
Get:4 http://ca.archive.ubuntu.com/ubuntu focal/universe amd64 libglade2.0-cil amd64 2.12.40-3 [16.8 kB]
Get:5 http://ca.archive.ubuntu.com/ubuntu focal/universe amd64 bless all 0.6.0-7 [406 kB]
Fetched 957 kB in 0s (4,123 kB/s)
Selecting previously unselected package libglade2-0:amd64.
(Reading database ... 186271 files and directories currently installed.)
Preparing to unpack .../libglade2-0_1%3a2.6.4-2ubuntu2_amd64.deb ...
Unpacking libglade2-0:amd64 (1:2.6.4-2ubuntu2) ...
Selecting previously unselected package libglib2.0-cil.
Preparing to unpack .../libglib2.0-cil_2.12.40-3_amd64.deb ...
Unpacking libglib2.0-cil (2.12.40-3) ...
Selecting previously unselected package libgtk2.0-cil.
Preparing to unpack .../libgtk2.0-cil_2.12.40-3_amd64.deb ...
Unpacking libgtk2.0-cil (2.12.40-3) ...
Selecting previously unselected package libglade2.0-cil.
Preparing to unpack .../libglade2.0-cil_2.12.40-3_amd64.deb ...
Unpacking libglade2.0-cil (2.12.40-3) ...
Selecting previously unselected package bless.
Preparing to unpack .../archives/bless_0.6.0-7_all.deb ...
Unpacking bless (0.6.0-7) ...
Setting up libglib2.0-cil (2.12.40-3) ...
* Installing 1 assembly from libglib2.0-cil into Mono
Setting up libglade2-0:amd64 (1:2.6.4-2ubuntu2) ...
Setting up libgtk2.0-cil (2.12.40-3) ...
* Installing 5 assemblies from libgtk2.0-cil into Mono
Processing triggers for sgml-base (1.29.1) ...
Setting up libglade2.0-cil (2.12.40-3) ...
* Installing 1 assembly from libglade2.0-cil into Mono
Processing triggers for desktop-file-utils (0.24-1ubuntu3) ...
Processing triggers for mime-support (3.64ubuntu1) ...
Processing triggers for gnome-menus (3.36.0-1ubuntu1) ...
Processing triggers for libc-bin (2.31-0ubuntu9.9) ...
Processing triggers for man-db (2.9.1-1) ...
Setting up bless (0.6.0-7) ...
```

downloading of bless

```
student@cr450:~/labs$ yara xorsigs.yar unknown -s
XorDos unknown
0x4e:$_13_d: 59 65 64 7E 2D 7D 7F 62 6A 7F 6C 60 2D
student@cr450:~/labs$
```

Variables & decimals obtained after the command above

XOR selected, d in the Hexadecimal & execute (results in the picture above)

Exercise #3 : Validation of sending an e-mail (10p)

An SMTP server with the IP address 142.116.34.182 sends an email according to the following parameters:

Email details:

Sending server IP address: 142.116.34.182
Sender's email address: dave.munger@polymtl.ca
Recipient's email address: dave.munger@cr450-polymtl.ca

Carry out the required analyses and answer the following questions:

A DNS resolution was required on the recipient's address to identify its hosts. After obtaining the host name, it is then possible to perform an A-type DNS resolution to get its IP address.

A. What are the host name(s) and IP address(es) of the server(s) to which the email must be sent in order to be received by the recipient? (**2p**)
    a. Host names: cr450-polymtl-ca.mail.protection.outlook.com
    b. IP address(es) : the mail server's IP address is 104.47.57.34

B. Will the destination server(s) (identified above) accept the email without restriction? (**8p**)

This may depend on several factors. It can depend on the security policies implemented by the administrators of the domain's mail system. However, the fact that the email is sent from a valid IP address and from the "same domain" may help reduce the risk that the email will be considered spam or blocked. It may also depend on which domains are considered "trusted" by the polymtl.ca domain. That said, the email should reach its destination without significant restrictions.

Exercise #4 : DKIM (10p)

To complete this exercise, you must receive an email originating from GMAIL. Use screenshots to document all the steps.

Carry out the required analyses and answer the following question:

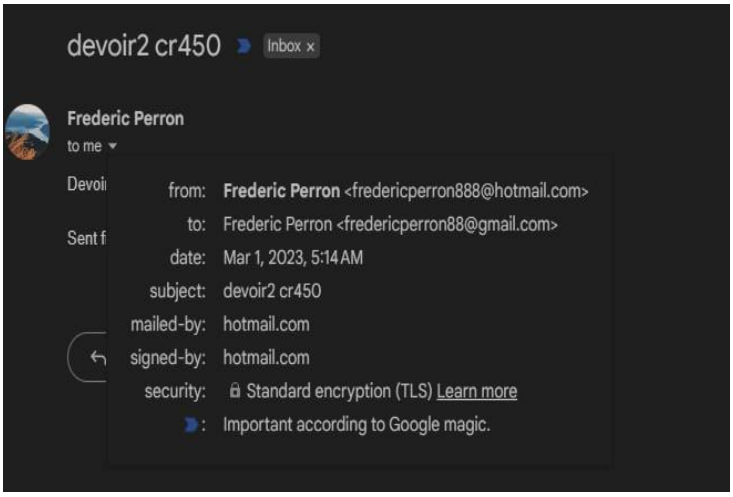
A. Identify the selector used by GMAIL. (**5p**)

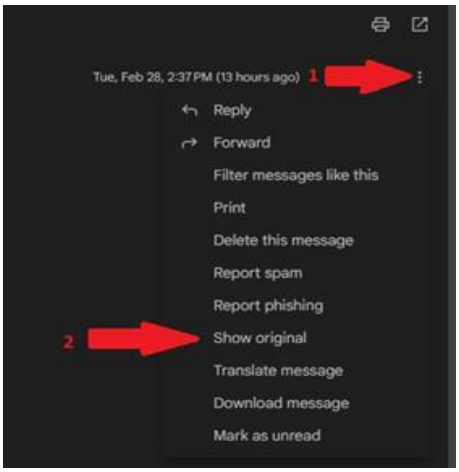
The selector used by GMAIL is arc-20160816

I started by sending myself an email from my Gmail address. Once this was done, I was able, by clicking on "Show original," to see the selector in the ARC-Message-Signature section. I then noted the domain (google.com) and the selector (arc-20160816). To model the command, I followed the nslookup Word document from Course 4. *See the steps in the screenshots below.*

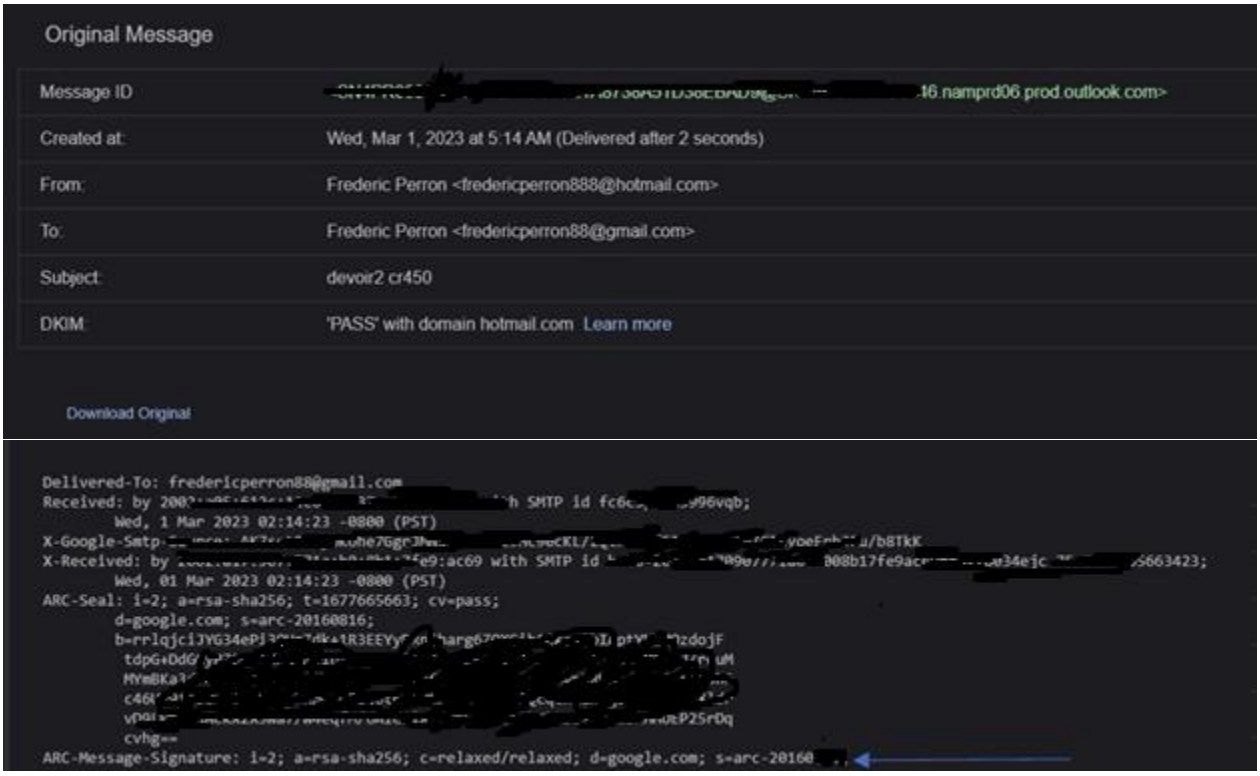
I first started by sending myself an email to my GMAIL (fredericperron88gmail.com)

Inbox Gmail





Steps to be able to click on "Show Original" of my received email



Results after I did ''Show Original''

We can see in the image above that the selector used is **arc-20160816** (see blue arrow) and that the domain is **google.com**.

Based on the Nslookup section from Course 4, I executed the following command: **nslookup > arc-20160816**. I then followed the course instructions and replaced *selector* and *domain* with their respective values/names. In other words, I replaced **selector._domainkey.domain.com** with **arc-20160816** (selector) and **google.com** (domain). This produced something like: **arc-20160816._domainkey.google.**

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\frede>nslookup
Default Server:  mynetwork.home
Address:  192.168.2.1

> set type=txt
```

results of the command to find the DKIM key

B. Document the details of the DKIM key in the DNS. (**5p**)

Arc-20160816._domainkey.google.com.

Exercise #5 – SiLK (10p)

WARNING! For each item, you must include a screenshot of your command. Finally, you must complete all exercises using your personal user account on the Security Onion platform.

Exercise 5 relies on the use of SiLK. Use the course notes and labs to help you.

a) Interrogate the assignment's .silk files (e.g., rwfilter *.silk) to determine the total number of flows observed between September 5 and 10, 2003, across all files. (5p)

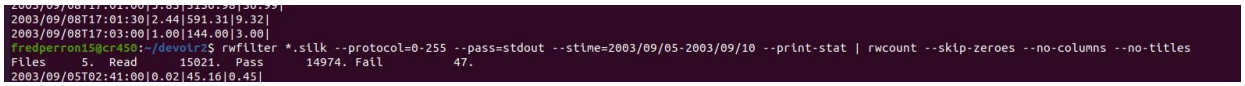
Hint

rwfilter is the best tool to answer this question. It has an option that quickly reports the number of aggregated flows based on your selection criteria.

Hint

The options --print-statistics indicate the number of files that had to be read from the repository, the number of flows read, the number that pass your filter, and the number of flows that fail your filter.

I began by using the command shown in class and selected the desired criteria. I had to add **no-columns** and **skip-zeroes**, as seen on a page found on Google (see references). Therefore, the number of flows corresponds to the number of "pass," which is **14,974**.



b) Please query the .silk files for flows that occurred between September 5 and 10, 2003. What is the total number of TCP flows present across all files? (5p)

Hint

You can answer this question using almost exactly the same command as in the previous exercise; only the filtering criteria to select the TCP protocol are missing.

You can specify a protocol to filter on using the **--protocol=** option.

I used the same command as before, except that I changed the **bin-size** to 60 and removed the rwcount filters. I also added only protocol **6** for TCP, which revealed **10,935 pass** (total number of TCP flows).

```
fredperron15@cr450:~/devoir2$ rwfilter *.silk --protocol=6 --pass=stdout --stime=2003/09/05-2003/09/10 --print-stat | rwcount --bin-size=60
Files      5.  Read       15021.  Pass       10935. Fail         4086.
               Date|         Records|               Bytes|           Packets|
```

Exercise #6 – SiLK (15p)

Statistics

SiLK provides the **rwstats** tool as a quick way to process flows as datasets. It can be used to rapidly return statistical information about the data in the repository and, depending on your query parameters, it can be grouped and organized in any way you prefer.

a) Use the tools rwfilter and rwstats to determine the ten main protocols that were used between September 5 and 10, 2003. (5p)

Hint : rwfilter

To answer this question, you will need to use **pass=stdout** with a filter as arguments to rwfilter, and then process the output through rwstats. Can you think of a filter you can use to see all protocols in the repository during this time period?

Hint : rwfilter

A simple filter you can use to examine all flows during a given period is **--proto=0-255**. This option allows you to select individual protocol lists separated by commas or protocol ranges. In this case, it will scan all IP flows for all protocols since all protocols will match.

Hint : rwstats

To use rwstats, you must specify a field or a set of **--fields** to group by, from which statistics will be produced. You must also specify a **--count** for the number of items you want reported. In this case, we are looking for the top ten items.

Flows, bytes, and packets

The rwstats tool reports the number of flows/streams by default. You can override this behavior using the **--bytes** or **--packets** option. The statistics will then be reported based on the selected criteria rather than the number of flows.

This can be very useful. For example, we may be more interested in the total number of bytes transferred between two hosts than in the number of flows observed. This is a good way to detect exfiltration spread across multiple connections, ports, and protocols.

To begin, I first noted the commands provided in the hints above and searched Google to learn more (see references). The command used is as follows:

```
Use 'rwfilter --help' for usage
fredperron15@cr450:~/devoir2$ rwfilter *.silk --protocol=0-255 --pass=stdout --stime=2003/09/05-2003/09/10 --print-stat | rwstats --top --field=protocol,sip --count=10
Files     5.  Read      15021.  Pass      14974. Fail          47.
INPUT: 14974 Records for 2876 Bins and 14974 Total Records
OUTPUT: Top 10 Bins by Records
pro|                                sIP|   Records|  %Records|   cumul_%|
  6|                           192.168.1.3|      5462| 36.476559| 36.476559|
 17|                           192.168.1.3|      2062| 13.770536| 50.247095|
 17|                       145.238.110.68|       260|  1.736343| 51.983438|
 17|                        134.214.100.6|       253|  1.689595| 53.673033|
 17|                        193.49.205.19|       247|  1.649526| 55.322559|
  6|                       200.184.43.197|       172|  1.148658| 56.471217|
  1|                           192.168.1.3|       164|  1.095232| 57.566449|
  6|                         63.243.90.10|       135|  0.901563| 58.468011|
  6|                          78.68.74.84|        93|  0.621077| 59.089088|
  6|                        203.248.234.10|        67|  0.447442| 59.536530|
```

We can see the top 10 protocols used in the left column labeled "pro."

b) In the .silk files, which source IP address transferred the most bytes between September 5 and 10, 2003? (5p)

Hint

The only real difference between this question and the previous exercises is that you must adjust the timestamps and look at the number of bytes rather than the number of flows.

The only modification required was to change the count/value to bytes, as shown on my SiLK command reference page. I therefore added **value=bytes** at the end of the rwstats command, which allowed me to obtain, in descending order, the source IPs with the highest number of bytes transferred. See the image below for the command :

```
fredperron15@cr450:~/devoir2$ rwfilter *.silk --protocol=0-255 --pass=stdout --stime=2003/09/05-2003/09/10 --print-stat | rwstats --top --field=sip,sport,dip,dport,protocol --count=10 --value=bytes
Files     5.  Read      15021.  Pass      14974. Fail          47.
INPUT: 14974 Records for 7839 Bins and 2912498 Total Bytes
OUTPUT: Top 10 Bins by Bytes
                          sIP|sPort|                            dIP|dPort|pro|              Bytes|    %Bytes|   cumul_%|
                 65.113.119.134|   80|                   192.168.1.3| 1043|  6|             454274| 15.597401| 15.597401|
                    192.168.1.3|  514|                 192.168.1.254|  514| 17|             208588|  7.161825| 22.759226|
                    192.168.1.3|  138|                 192.168.1.255|  138| 17|             190155|  6.528932| 29.288157|
                    192.168.1.3|  123|                 193.49.205.19|  123| 17|              34428|  1.182078| 30.470236|
                    192.168.1.3|  123|                  134.214.100.6|  123| 17|              33212|  1.140327| 31.610562|
                    192.168.1.3|  123|                145.238.110.68|  123| 17|              32908|  1.129889| 32.740452|
                  134.214.100.6|  123|                   192.168.1.3|  123| 17|              30552|  1.048996| 33.789448|
                 145.238.110.68|  123|                   192.168.1.3|  123| 17|              30172|  1.035949| 34.825397|
                  193.49.205.19|  123|                   192.168.1.3|  123| 17|              30172|  1.035949| 35.861347|
                    192.168.1.3|  443|                 61.61.123.123|33587|  6|              29286|  1.005529| 36.866875|
fredperron15@cr450:~/devoir2$
```

The IP address that transferred the most bytes between September 5 and 10 is **65.113.119.134**.

c) Between September 5 and 10, 2003, at what time did the flow that transferred the most bytes start? (5p)

Hint

This is a more mentally challenging question. When we pass a set of fields to rwstats, it uses those fields to create bins—groupings of data. For example, if we pass the following:

--fields sip,dip

This would report the highest number of flows where the source IP address and destination IP address in the flows are used as keys, regardless of ports or protocols. If we modify this slightly:

--fields sip,dip --bytes

This would now report the source-to-destination address pair that had the greatest number of bytes in the aggregate rather than the greatest number of flows.

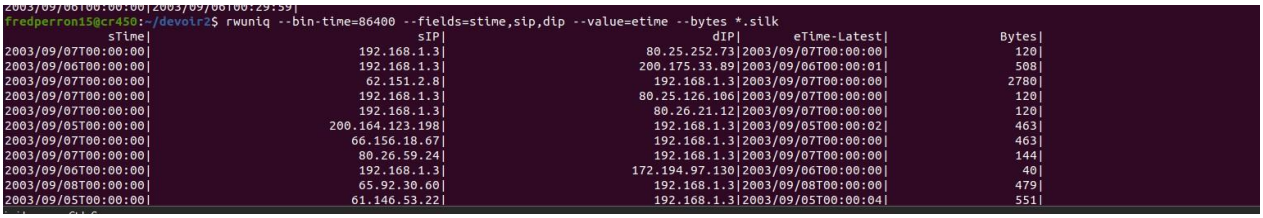
--fields sip,dport

This would now aggregate all flows by source IP address and destination port only, reporting the pairing that has the highest number of flows.

For this exercise, I simply used **rwuniq** with the fields set to **stime,sip,dip**, the value set to **etime**, and counting bytes. This resulted in a command such as:

**rwuniq --bin-time=86400 --fields=stime,sip,dip --value=etime --bytes .silk,*

which produced a list sorted by bytes. I then scrolled down until I found the exchange with the highest number of bytes, which was **463,812**, sent at **00:00** and received at **00:17**.



```
fredperron15@cr450:~/devoirz$ rwuniq --bin-time=86400 --fields=stime,sip,dip --value=etime --bytes *.silk
              sTime|              sIP|              dIP|      eTime-Latest|  Bytes|
2003/09/07T00:00:00|     192.168.1.3|  80.25.252.73|2003/09/07T00:00:00|    120|
2003/09/06T00:00:00|     192.168.1.3| 200.175.33.89|2003/09/06T00:00:01|    508|
2003/09/07T00:00:00|      62.151.2.8|   192.168.1.3|2003/09/07T00:00:00|   2780|
2003/09/07T00:00:00|     192.168.1.3| 80.25.126.106|2003/09/07T00:00:00|    120|
2003/09/07T00:00:00|     192.168.1.3|  80.26.21.12|2003/09/07T00:00:00|    120|
2003/09/05T00:00:00| 200.164.123.198|   192.168.1.3|2003/09/05T00:00:02|    463|
2003/09/07T00:00:00|    66.156.18.67|   192.168.1.3|2003/09/07T00:00:00|    463|
2003/09/07T00:00:00|     80.26.59.24|   192.168.1.3|2003/09/07T00:00:00|    144|
2003/09/06T00:00:00|     192.168.1.3| 172.194.97.130|2003/09/06T00:00:00|     40|
2003/09/08T00:00:00|     65.92.30.60|   192.168.1.3|2003/09/08T00:00:00|    479|
2003/09/05T00:00:00|    61.146.53.22|   192.168.1.3|2003/09/05T00:00:04|    551|
```

Command used

```
2003/09/06T00:00:00|     80.25.21.53|   192.168.1.3|2003/09/06T00:00:00|    144|
2003/09/08T00:00:00|  65.113.119.134|   192.168.1.3|2003/09/08T00:00:17|  463812|
```

Exchange with the biggest number of bytes (hour indicated)

Exercise #7 - Suricata(25p)

WARNING! For each requested rule, you must include a screenshot of your rule as well as a screenshot of the alert generated by it. In addition, your first and last name must always be included in the alert. Finally, you must complete all exercises using your personal user account on the Security Onion platform.

Use the pcap file “01_exploit.pcap” as input for this exercise. The objective of this lab is to create a functional signature that successfully detects the exploit used to compromise a system based on the following fictitious CVE and the related packet. Consider the provided packet capture as an example collected while running the proof-of-concept exploit against a LamanServer service on the local network.

CVE- 2021-0503¶

Description Une condition de dépassement de mémoire tampon entraînant l'exécution de code à distance a été identifiée dans le service Enterprise LamanServer , qui s'exécute généralement sur le port TCP 50503. Bien qu'une preuve de concept ait été démontré e, l'exploitation n'a pas encore été observée dans la nature. La condition de dépassement de mémoire tampon existe dans l'analyseur de commande de protocole, qui n'est accessible qu'après l'envoi réussi d'un message HELO.

References

Note : References are provided for the reader's convenience to help distinguish vulnerabilities. The list is not intended to be exhaustive.

BID: 503001
URL: http://www.securityfocus.com/SMTP503001

Description : Create a Suricata rule that successfully detects the exploit found in 01_exploit.pcap using one or more content options.

a) Create a simple alert rule that uses only a rule header and verify its function using the file 01_exploit.pcap, displaying the alerts on the console. Your rule must alert on every packet found in the file. (5p)

To create a ruler header, you must first decide on the rule action. It can be one of the following:

- **alert** - Generate an alert
- **pass** - Allow this traffic without applying other actions.
- **log** - Log the associated packets but do not generate an alert.
- **sdrop** - IPS only. Block the packet and silently drop it.
- **Ddrop -** IPS only. Block the packet and log it.
- **reject -** Block the packet, log it, and generate a TCP RST to terminate the connection.
- **activate -** Activate a dynamic rule that should become active if this rule matches.
- **dynamic** - Specify that this is a dynamic alert rule.

Hint

All rule headers must specify a protocol, a source, a source port, a direction, a destination, and a destination port. Valid protocols include:

- IP

- TCP
- UDP
- ICMP

Arbitrary IP protocols are not supported. The general structure of a rule header is:

```
alert IP $SOURCE $SPORT -> $DEST $DPORT
```

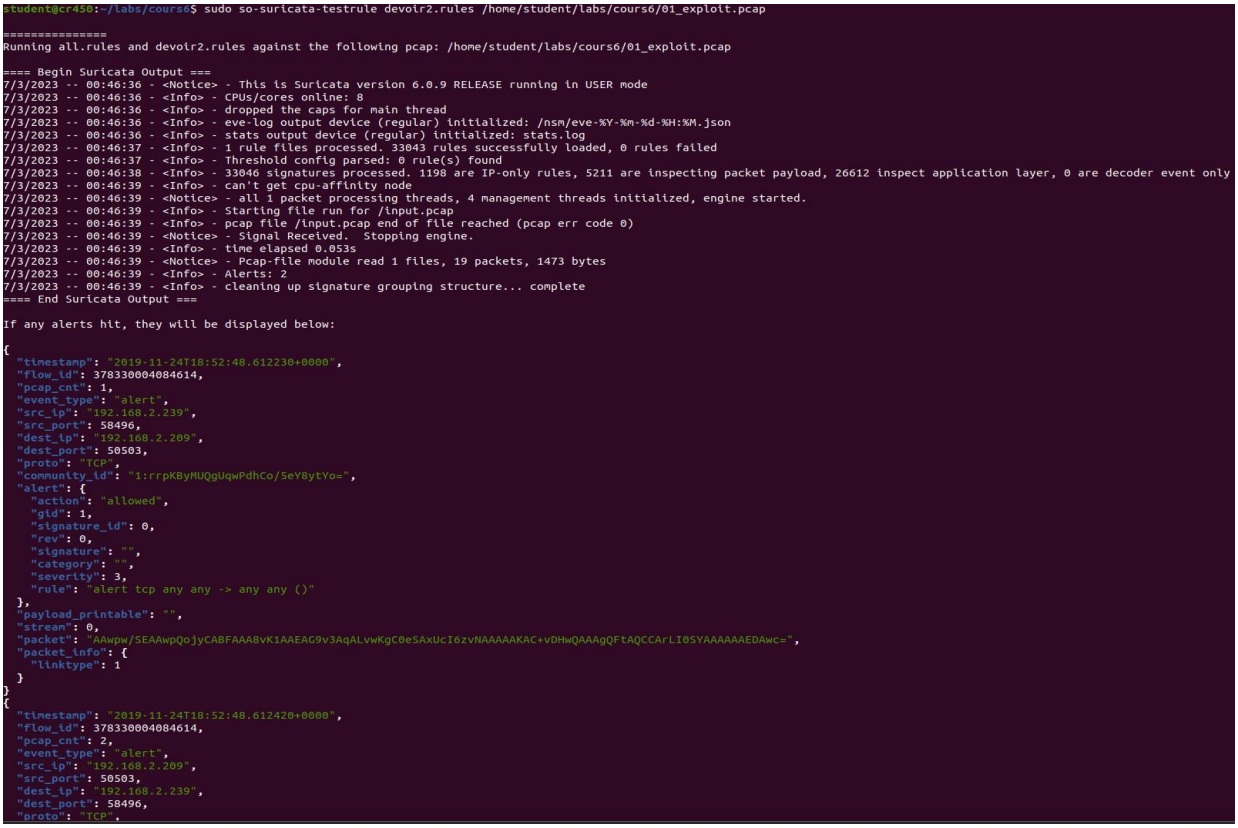
I began by creating a file named **devoir2.rules** for my Suricata rules.

```
student@cr450:~/labs/cours6$ nano cours6.rules
```

Next, I entered a very basic rule to generate an alert for each packet.

```
  GNU nano 4.8
alert tcp any any -> any any ()
```

I then ran the Suricata command using my selected **.rules** file along with the assignment's Wireshark file, and the alerts appeared. *See the image below*



b) Add metadata to your rule. Use SID 1000000 as the signature ID. Make sure to include a revision number of 1 and an alert message of your choice. The BugTraq and CVE metadata must match the information from the CVE above. After modifying your rule, verify that it still alerts correctly using Suricata. (5p)

I modified the Suricata rule to include metadata, an alert message, a signature ID, etc. Here is the nano file I used as a reference to proceed:

```
  GNU nano 4.8
alert tcp any any -> any any (msg: " tcp alert cr450"; sid:1000001; rev:1; classtype:tcp-event;)
```

I then ran the Suricata command to generate alerts with my messages. *See the image below*

```
student@cr450:~/labs/course6$ sudo so-suricata-testrule devoir2.rules /home/student/labs/course6/01_exploit.pcap
[sudo] password for student:

==============
Running all.rules and devoir2.rules against the following pcap: /home/student/labs/course6/01_exploit.pcap

==== Begin Suricata Output ===
7/3/2023 -- 01:09:06 - <Notice> - This is Suricata version 6.0.9 RELEASE running in USER mode
7/3/2023 -- 01:09:06 - <Info> - CPUs/cores online: 8
7/3/2023 -- 01:09:06 - <Info> - dropped the caps for main thread
7/3/2023 -- 01:09:06 - <Info> - eve-log output device (regular) initialized: /nsm/eve-%Y-%m-%d-%H:%M.json
7/3/2023 -- 01:09:06 - <Info> - stats output device (regular) initialized: stats.log
7/3/2023 -- 01:09:06 - <Warning> - [ERRCODE: SC_ERR_UNKNOWN_VALUE(129)] - signature sid:1000001 uses unknown classtype: "tcp-event", using default priority 3. This message won't be shown again for this classtype
7/3/2023 -- 01:09:06 - <Info> - 1 rule files processed. 33043 rules successfully loaded, 0 rules failed
7/3/2023 -- 01:09:06 - <Info> - Threshold config parsed: 0 rule(s) found
7/3/2023 -- 01:09:07 - <Info> - 33046 signatures processed. 1198 are IP-only rules, 5211 are inspecting packet payload, 26612 inspect application layer, 0 are decoder event only
7/3/2023 -- 01:09:08 - <Info> - can't get cpu-affinity node
7/3/2023 -- 01:09:08 - <Notice> - all 1 packet processing threads, 4 management threads initialized, engine started.
7/3/2023 -- 01:09:08 - <Info> - Starting file run for /input.pcap
7/3/2023 -- 01:09:08 - <Info> - pcap file /input.pcap end of file reached (pcap err code 0)
7/3/2023 -- 01:09:08 - <Notice> - Signal Received.  Stopping engine.
7/3/2023 -- 01:09:08 - <Info> - time elapsed 0.062s
7/3/2023 -- 01:09:08 - <Notice> - Pcap-file module read 1 files, 19 packets, 1473 bytes
7/3/2023 -- 01:09:08 - <Info> - Alerts: 2
7/3/2023 -- 01:09:09 - <Info> - cleaning up signature grouping structure... complete
==== End Suricata Output ===

If any alerts hit, they will be displayed below:

{
  "timestamp": "2019-11-24T18:52:48.612230+0000",
  "flow_id": 1816409608836998,
  "pcap_cnt": 1,
  "event_type": "alert",
  "src_ip": "192.168.2.239",
  "src_port": 58496,
  "dest_ip": "192.168.2.209",
  "dest_port": 50503,
  "proto": "TCP",
  "community_id": "1:rrpKByMUQgUqwPdhCo/5eY8ytYo=",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 1000001,
    "rev": 1,
    "signature": " tcp alert cr450",
    "category": "",
    "severity": 3,
    "rule": "alert tcp any any -> any any (msg: \" tcp alert cr450\"; sid:1000001; rev:1; classtype:tcp-event;)"
  },
  "payload_printable": "",
  "stream": 0,
  "packet": "AAwpw/SEAAwpQojyCABFAAA8vK1AAEAG9v3AqALvwKgC0eSAxUcI6zvNAAAAAKAC+vDHwQAAAgQFtAQCCArLI0SYAAAAAAEDAwc=",
  "packet_info": {
    "linktype": 1
  }
}
{
  "timestamp": "2019-11-24T18:52:48.612420+0000",
  "flow_id": 1816409608836998,
  "pcap_cnt": 2,
  "event_type": "alert",
  "src_ip": "192.168.2.209",
  "src_port": 50503,
```

Meta data of rule

As soon as you add rule options, you must include certain metadata. Although the SID is required, most metadata is not. Nevertheless, it is recommended to include several elements:

- rev - You must increment the revision metadata value each time you modify your rule. This will save you a lot of time and effort when troubleshooting rule changes. It is common for the rule you edit not to be the one actually in use; this option helps detect that sooner.

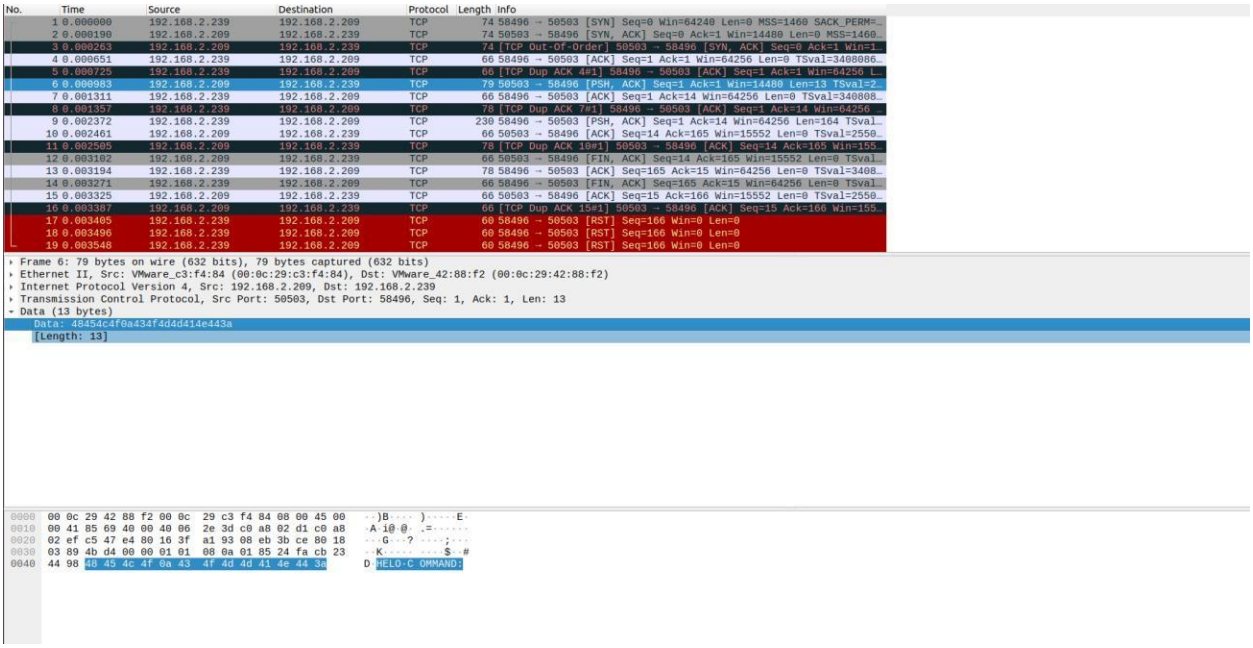
- reference - Allows you to specify metadata for CVE, BugTraq, Nessus, Arachnids, McAfee, OSVDB, MSB, or raw URLs. When using a supported reference, it will automatically be converted into a URL. More information can be found in the online Suricata manual, section 3.4.2.

- sid - The SID is mandatory. Remember that, from your perspective, all numbers < 1,000,000 are reserved.

- msg - An arbitrary string that will be included as the main alert message.

a. Examine the packets provided in 01_exploit.pcap. Identify content that could potentially be used in a signature match that would not be likely to generate a false positive. (5p)

Corresponding content



When identifying content to use in a rule, you should look for data that is unlikely to appear commonly in normal network traffic. In general, a string of at least four bytes—preferably longer—is recommended. If shorter strings or byte sequences must be specified, always try to include a longer string that must also be present.

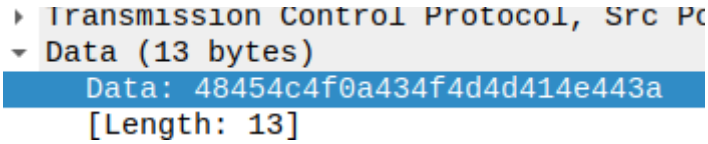
Suricata will always attempt to find the longest string that is either at a known offset or can be found relative to the start of the packet payload.

Hint

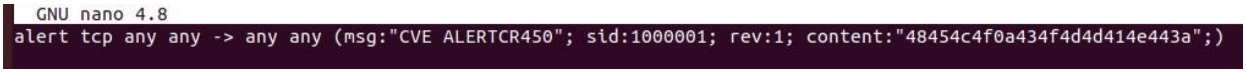
Use any tool you are comfortable with to view packet payloads. Wireshark or tcpdump are good choices.

c) Use the content you identified in the previous question along with protocol information to refine your rule. After doing this, verify that Suricata alerts correctly on this exploit attempt. (10p)

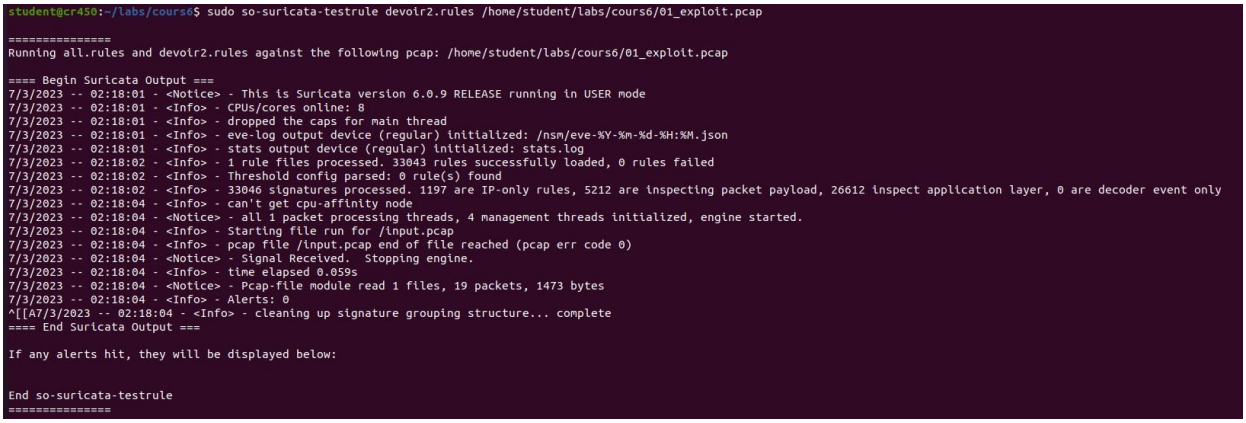
This can be done simply by adding the packet's data as a **content** option in my Suricata rule.



data string of packet in question



suricata rule



command execution

REFERENCES :

https://tools.netsa.cert.org/silk/silk-reference-guide.html#x1-1830001

https://tools.netsa.cert.org/silk/silk-reference-guide.html